SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	**Subash kannan.R**	**Reg.No.:**	**192511237**
Department:	**BE.CSE**	**Date:**	**16/082026**

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM,

DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

***Code of Conduct**:*

*I **Subash kannan.R(192511237)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	**Application of Engineering Concepts (25%)**	**Solution Design (25%)**	**Use of Tools (15%)**	**Analysis (10%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

ANNEXURE A

Industry Problem Solving Task

Q1. Smartphone Performance Optimization
Scenario: A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM, and propose a hierarchical memory redesign to reduce latency and improve throughput.

1. Understanding of the Industry Problem

App-switch lag on a smartphone SoC typically stems from cold cache/TLB state and DRAM page-table reloads each time the OS swaps the foreground process's working set. The constraint set includes a tight power/thermal budget, limited die area for cache, and mobile LPDDR DRAM bandwidth that is far lower than desktop DRAM. The core problem is that every app switch forces a near-total cache flush and re-population, exposing the full L1→L2→L3→DRAM latency chain repeatedly.

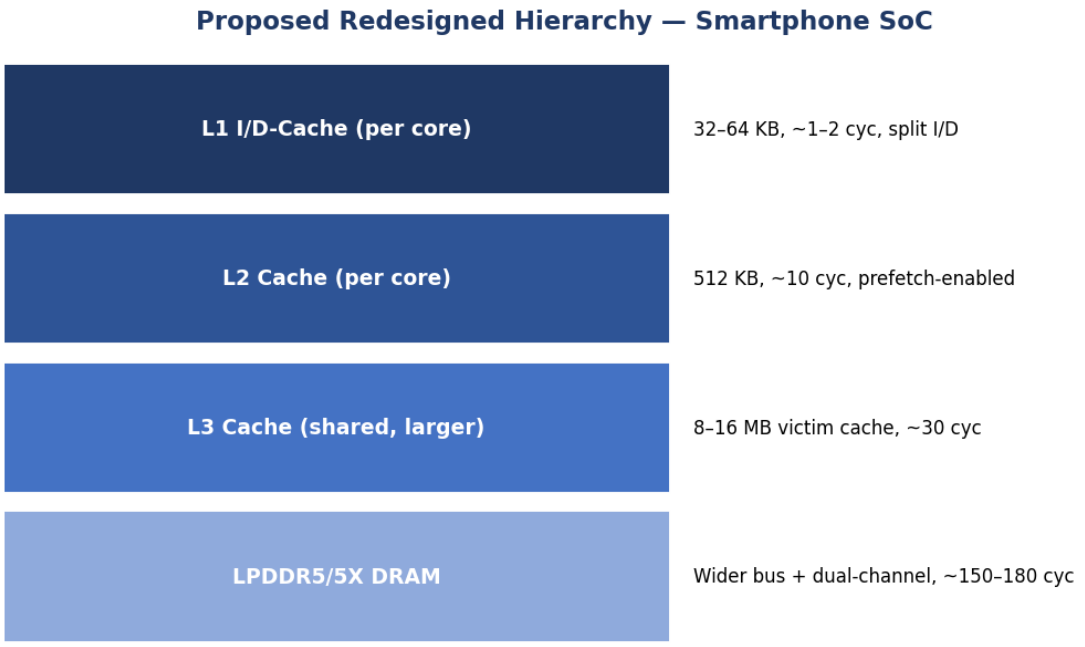
2. Application of Engineering Concepts

Memory Level	Latency	Bandwidth Impact	Role in App-Switch Lag
L1 Cache	~1–2 cycles	Very High	Serves per-core hot instructions/data; too small to hold multiple apps' working sets
L2 Cache	~10 cycles	High	Buffers per-core misses; still flushed on context switch
L3 Cache (shared)	~30 cycles	Moderate (shared)	Retains some data across context switches if large enough — key to reducing reload cost
DRAM (LPDDR)	~150–200 cycles	Low relative to cache	Every full cache miss chain ends here; narrow mobile bus limits throughput

3. Proposed Solution Design

Redesign the hierarchy so the shared L3 acts as a larger 'victim'/retention cache sized to hold the combined working sets of 2–3 recently used apps, add way-partitioning so a foreground app cannot evict the whole L3, and widen/duplex the LPDDR channel to raise

DRAM throughput for the residual misses. This shortens the effective miss chain from full DRAM round-trips to mostly L3 hits.



4. Use of Tools / Supporting Data (Visualization)

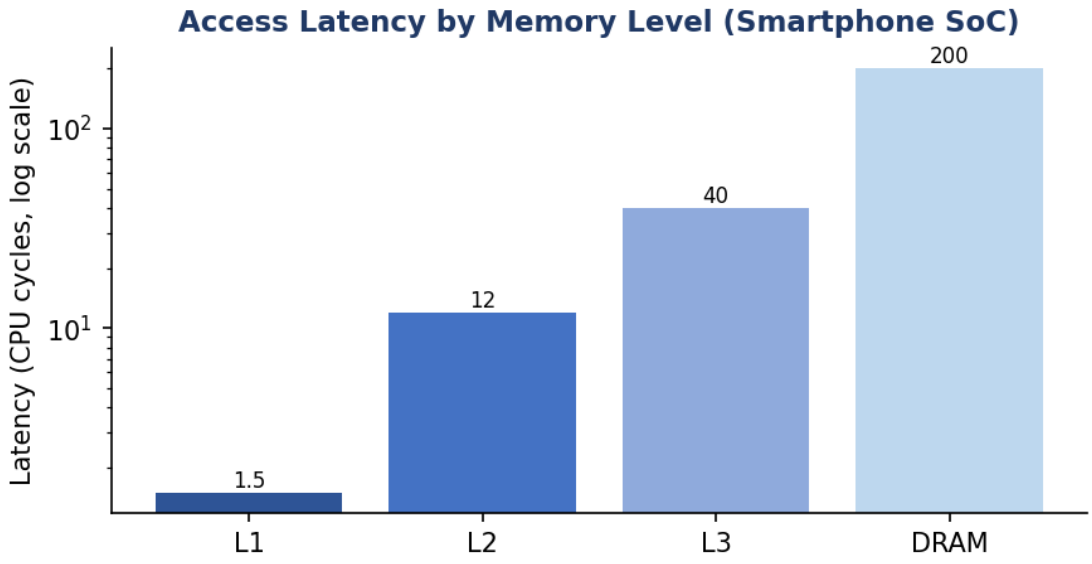


Fig. Q1 — Latency by memory level, current SoC hierarchy (log scale).

5. Analysis & Expected Outcome

With an enlarged, partitioned L3 (8–16 MB) and dual-channel LPDDR5X, the expected effect is a significant cut in average memory access time (AMAT) during app switches, since the majority of reloaded pages/instructions are served from L3 (~30 cycles) instead of DRAM (~200 cycles) — an estimated 5–6x latency reduction for the app-switch critical path, translating to smoother UI transitions within the same power budget.

Q2. Cloud Server Memory Bottleneck

Scenario: A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

1. Understanding of the Industry Problem

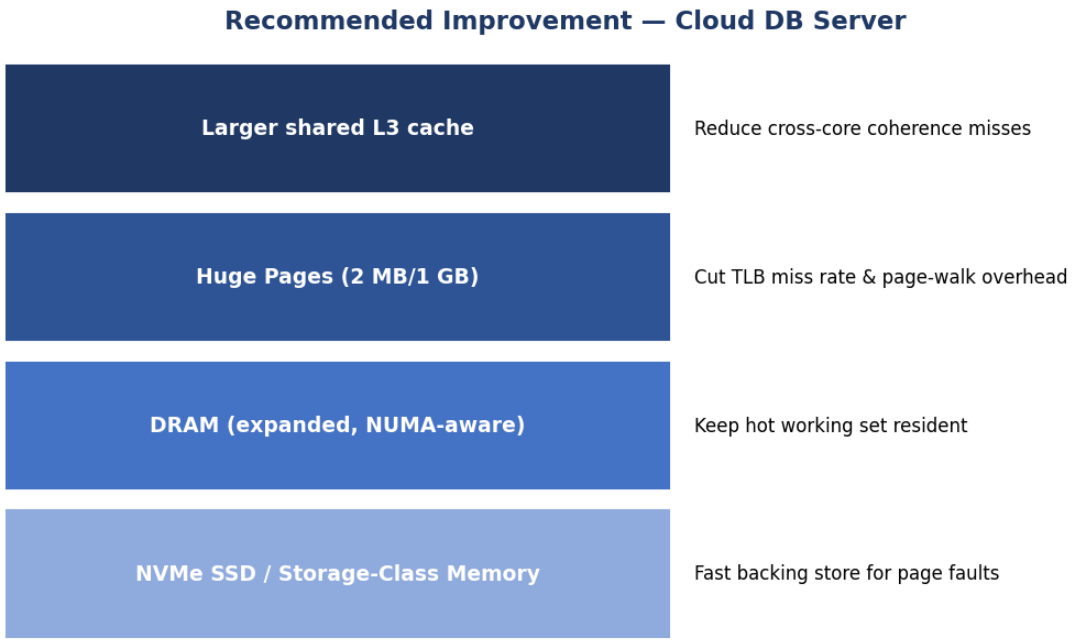
Database workloads on cloud servers have working sets far larger than any cache level, so queries constantly cross from the CPU cache hierarchy into the OS-managed virtual memory system. High query latency typically indicates frequent TLB misses, page-table walks, or — worst case — page faults that hit secondary storage. The problem must be understood at two levels: the hardware cache path (fast, small) and the software-managed paging path (large, orders of magnitude slower on a miss).

2. Application of Engineering Concepts

Mechanism	**Latency**	**Management**	**Effect on Query Latency**
L1/L2/L3 Cache Path	1–40 cycles	Hardware-managed	Serves hot rows/index pages already resident
TLB (address translation cache)	~1 cycle (hit)	Hardware, small	Avoids page-table walk for recently used pages
Page-Table Walk	~20–30 cycles	Triggered on TLB miss	Extra memory accesses to resolve virtual→physical address
Page Fault (paging to disk/SSD)	Microseconds to milliseconds	Software-handled, very slow	Dominates latency when working set exceeds RAM

3. Proposed Solution Design

Recommend increasing shared L3 capacity to reduce index/hot-row re-fetching, adopting huge pages (2 MB/1 GB) to cut TLB miss rate and page-walk overhead for large database buffer pools, and moving the paging backing store from spinning disk/SATA SSD to NVMe or storage-class memory (e.g., Optane-class or NAND-based SCM) so that unavoidable page faults are far less costly.



4. Use of Tools / Supporting Data (Visualization)

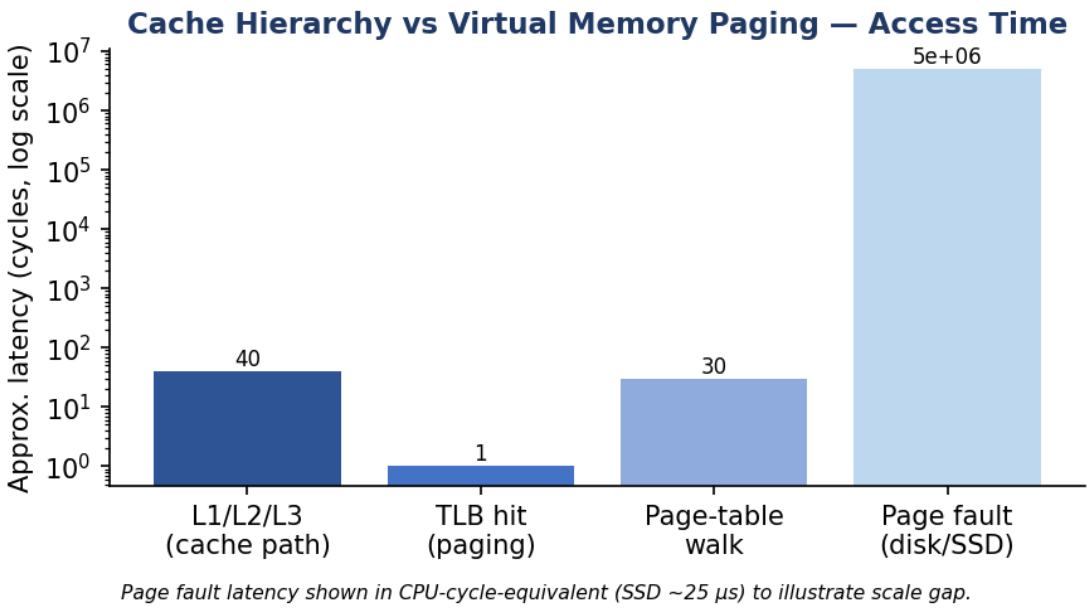


Fig. Q2 — Cache-hierarchy path vs virtual-memory paging path, approximate access time (log scale).

5. Analysis & Expected Outcome

Huge pages reduce the number of TLB entries needed to cover a large buffer pool, cutting TLB-miss-induced page-walks; combined with an NVMe-backed page store, worst-case page-fault latency drops by roughly one to two orders of magnitude versus SATA SSD/HDD. Net effect: query tail latency becomes dominated by cache/TLB behaviour rather than rare but extremely costly page faults.

Q3. Autonomous Vehicle Data Processing

Scenario: An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

1. Understanding of the Industry Problem

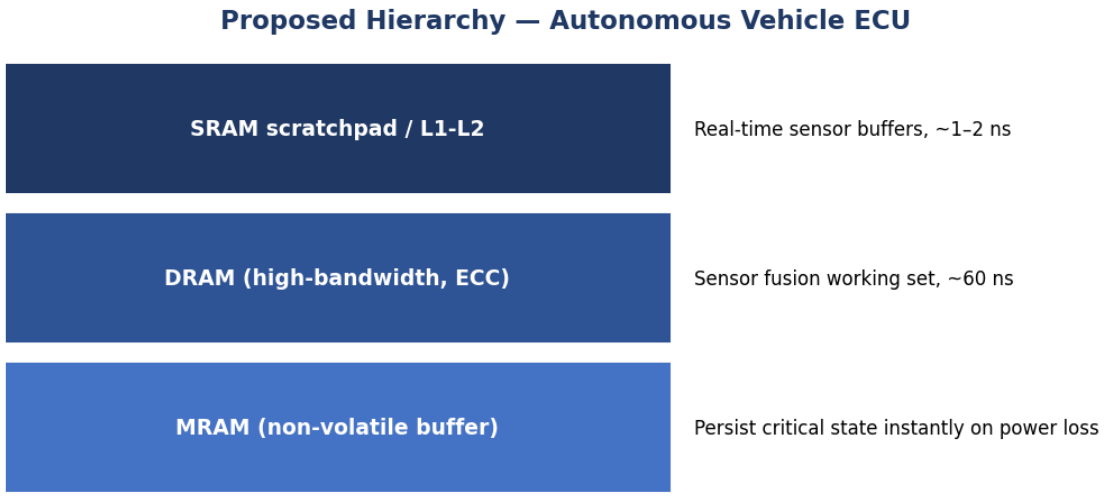
Autonomous-vehicle ECUs must fuse camera, LiDAR, and radar streams under hard real-time deadlines; any excessive latency or state loss on power interruption is safety-critical, not just a performance issue. The design constraint is therefore threefold: nanosecond-level determinism for control loops, high sustained bandwidth for sensor fusion, and non-volatility for critical state in case of power transients.

2. Application of Engineering Concepts

Technology	Latency	Bandwidth	Role in ECU Memory Hierarchy
SRAM	~1–2 ns	High (multi-port)	On-chip scratchpad/cache for control-loop and per-frame buffers
DRAM	~50–70 ns	High sustained (wide bus, ECC)	Sensor-fusion working set, frame buffers
MRAM	~5–10 ns	Moderate–High	Non-volatile store for critical state, instant-on after power loss

3. Proposed Solution Design

Use SRAM scratchpads for the hard-real-time control loop and per-frame sensor buffers (deterministic, no refresh stalls), ECC-protected DRAM with a wide bus for the sensor-fusion and perception working set, and MRAM as a non-volatile buffer that continuously mirrors critical safety state (e.g., last-known trajectory) so no data is lost on power interruption — avoiding the multi-millisecond penalty of falling back to flash/SSD.



4. Use of Tools / Supporting Data (Visualization)

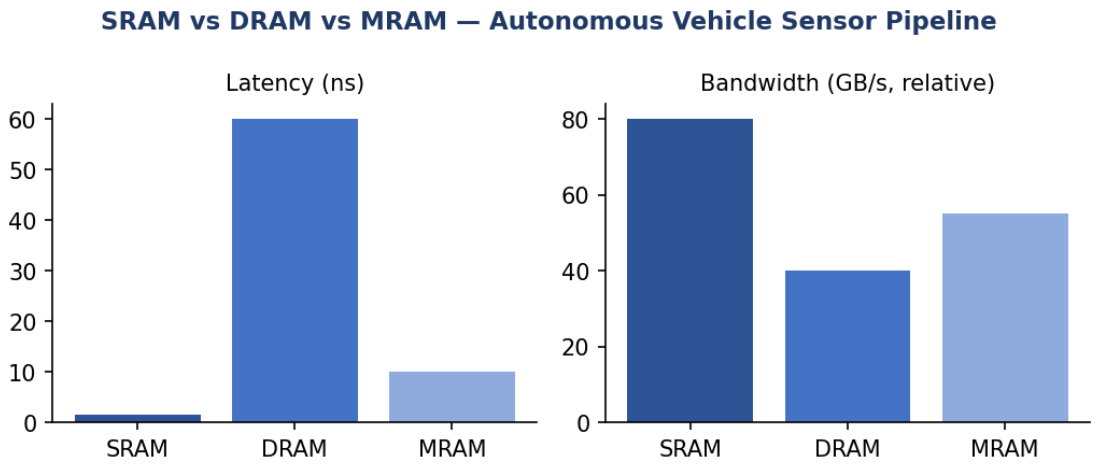


Fig. Q3 — Latency and relative bandwidth: SRAM vs DRAM vs MRAM.

5. Analysis & Expected Outcome

This hierarchy keeps the control-loop critical path entirely within SRAM/DRAM latencies (tens of nanoseconds), meeting real-time deadlines, while MRAM adds safety-relevant persistence with a latency penalty measured in nanoseconds rather than the milliseconds a flash-based checkpoint would cost — directly reducing the risk window during power anomalies.

Q4. AI Inference Edge Device

Scenario: An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

1. Understanding of the Industry Problem

Edge AI devices (e.g., battery-powered inference accelerators) must load trained model weights quickly on wake-up while keeping idle and active power low. NAND Flash offers dense non-volatile storage but with slow, block-oriented access; DRAM offers speed but is volatile and power-hungry to refresh continuously; the problem is to minimize both model-load latency and standby energy.

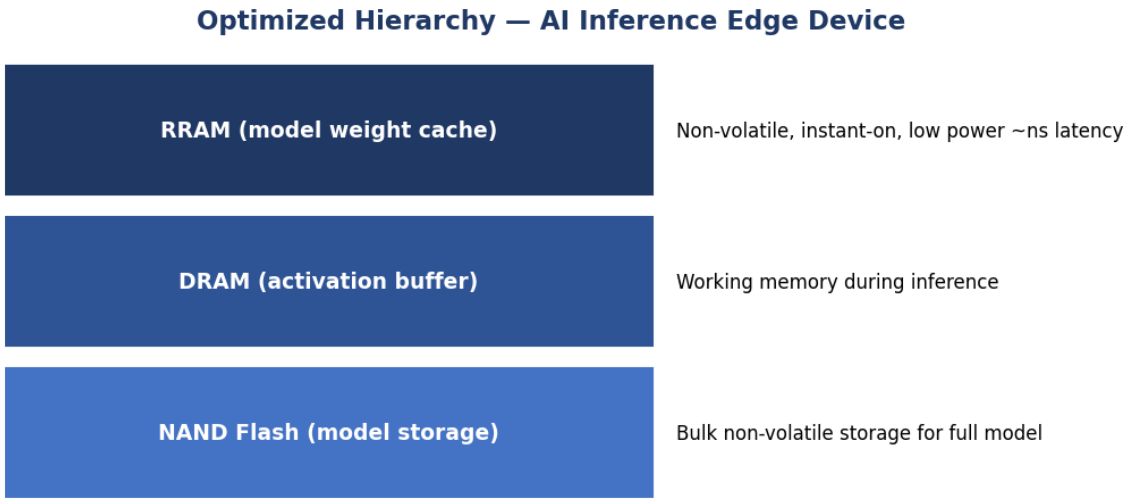
2. Application of Engineering Concepts

Technology	**Latency**	**Power/Volatility Profile**	**Role in Edge AI Hierarchy**
NAND Flash	~25–100 µs (read)	Non-volatile, dense, cheap per bit	Bulk storage for the full trained model

DRAM	~50–100 ns	Volatile, needs refresh power	Active working memory during inference
RRAM	~ns range	Non-volatile, low write energy	Fast-loading weight cache close to the accelerator

3. Proposed Solution Design

Store the full model in NAND Flash for cost-effective bulk persistence, but cache the frequently-used/most-recent model weights in RRAM so inference can begin almost instantly on wake without waiting on a full NAND read, and use a modestly-sized DRAM buffer only for transient activations — keeping DRAM refresh (and thus idle power) to a minimum.



4. Use of Tools / Supporting Data (Visualization)

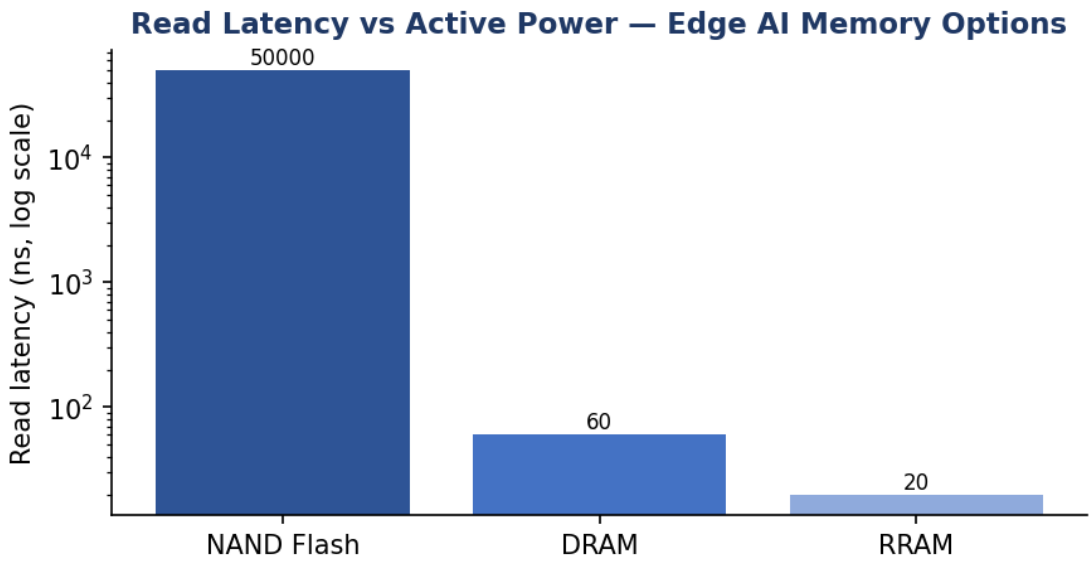


Fig. Q4 — Read latency comparison: NAND Flash vs DRAM vs RRAM (log scale).

5. Analysis & Expected Outcome

Because RRAM is non-volatile and needs no refresh, the device can power down DRAM between inference bursts without losing the cached weights, cutting idle power substantially versus keeping the model resident in DRAM. Model 'cold start' latency drops from NAND's tens-of-microseconds range to RRAM's nanosecond-scale access for the cached portion, directly improving wake-to-inference time.

Q5. High-Performance Gaming Console

Scenario: A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

1. Understanding of the Industry Problem

Large open-world scene loads stream textures, geometry, and audio assets faster than the DRAM/cache path can supply them, causing frame drops as the GPU stalls waiting on data. The bottleneck is throughput (sustained GB/s), not just latency: L3 cache offers very high bandwidth but limited capacity, while GDDR/DDR DRAM offers large capacity but comparatively lower sustained throughput under mixed CPU/GPU contention.

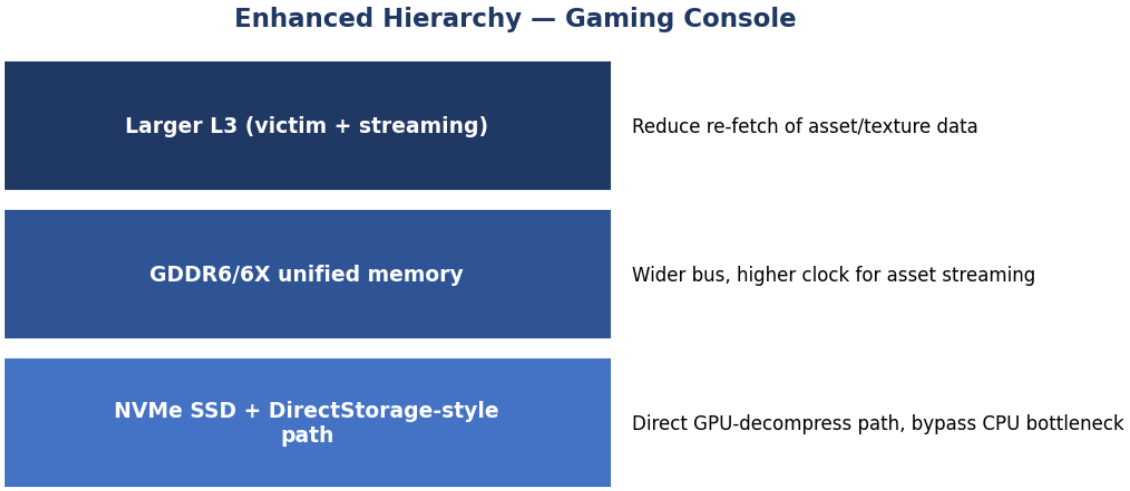
2. Application of Engineering Concepts

Memory Level	Throughput	Capacity Constraint

L3 Cache (shared)	~300–400 GB/s (on-die)	Small (tens of MB) — cannot hold full scene assets
GDDR6/DDR DRAM	~50–70 GB/s (shared bus)	Large (8–16+ GB) but bottlenecked under CPU+GPU contention

3. Proposed Solution Design

Increase effective streaming throughput by enlarging L3 to act as a texture/asset streaming buffer, widening/upgrading to a higher-clocked GDDR6X-class DRAM bus for peak scene-load bursts, and adding an NVMe SSD with a direct GPU-decompression path (bypassing the CPU) so raw asset throughput no longer depends solely on the DRAM bus during a load screen.



4. Use of Tools / Supporting Data (Visualization)

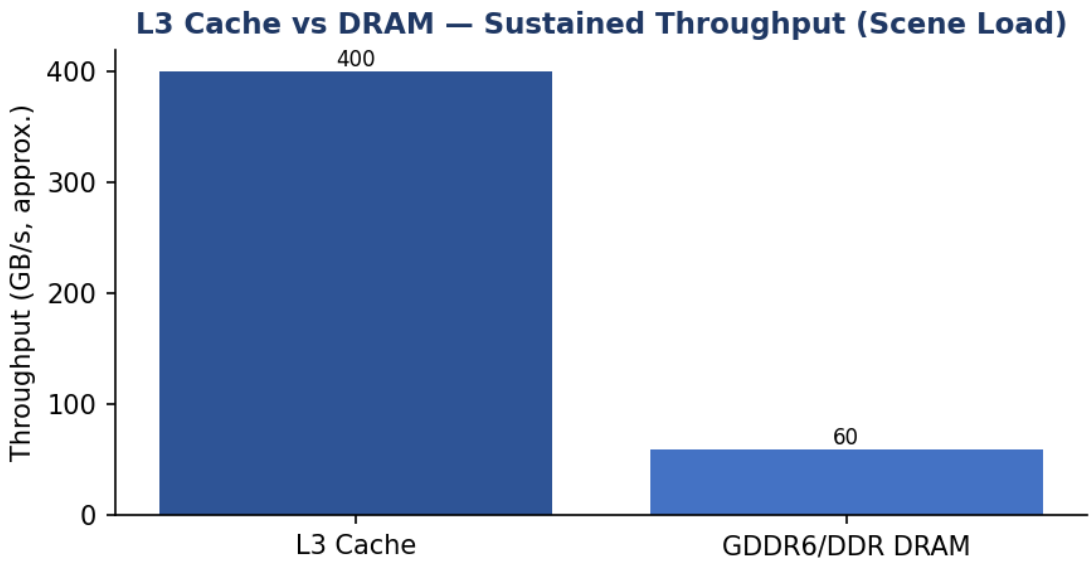


Fig. Q5 — Sustained throughput: L3 cache vs DRAM during scene loading.

5. Analysis & Expected Outcome

By offloading bulk asset transfer to a direct NVMe-to-GPU decompression path and using the enlarged L3 purely as a fast streaming buffer, DRAM bus contention during scene loads is reduced, allowing the GPU to receive assets at a more consistent rate and reducing the frame-time spikes that manifest as dropped frames.